Interactive Procedural City Generator with Layout Optimization

Pravin Game, Sheetal Chincharekar, Gaurangi Madrewar, Khushi Yuwanathe
Department of Computer Engineering, Pimpri Chinchwad College of Engineering, Pune, India

Abstract

Urban road connectivity influences the distance between emergency incidents and hospitals. This paper presents an implemented interactive two-dimensional procedural city generator combining deterministic road, block, and building generation with graph-based emergency accessibility evaluation and greedy road addition. Road intersections are represented as graph vertices; breadth-first search checks hospital reachability, and Dijkstra's algorithm measures shortest road-network distances. A normalized accessibility score combines 60% reachability and 40% distance efficiency. The system uses C++17, OpenGL 2.1, and Dear ImGui for interactive visualization and before/after comparison. We evaluated five reproducible seeds at fixed density: three improved in score, while two remained unchanged; all maintained 4/4 incident reachability. For seed 42, the score increased from 87.73 to 94.58 and mean road distance fell from 306.71 to 135.53 world units after four road additions. Twelve automated tests passed. Results demonstrate the potential of a lightweight educational graph-based city-layout tool, not a calibrated real-world emergency response model.

Index Terms—*Procedural city generation, computer graphics, road graphs, BFS, Dijkstra's algorithm, emergency accessibility, greedy optimization, 2D visualization.*

I. INTRODUCTION

Urban environments consist of roads, intersections, buildings, blocks, and facilities whose spatial arrangement influences connectivity. Designing and repeatedly evaluating alternative layouts manually is time-consuming. Procedural generation offers a reproducible way to explore different layouts, while graph algorithms allow quantitative analysis of the resulting road networks.

Parish and Müller [1] established an influential approach to procedural city modeling; Chen et al. [2] studied interactive street-network modeling. This work focuses on integrating generation, analysis, optimization, and visual comparison in a lightweight application. Rather than predicting actual ambulance travel time, it examines geometric road-network accessibility to hospitals in a simplified city.

The main contributions are (i) deterministic generation and finite comparison of alternative layouts; (ii) explicit BFS/Dijkstra analysis of weighted road graphs; (iii) a reproducible 60:40 accessibility score and greedy add-only optimization; (iv) a layered interactive 2D OpenGL visualization; and (v) five-seed before/after measurements with reported Debug computation timings and automated regression tests.

II. Literature Survey

Procedural city generation has been studied using different techniques to create road networks, buildings, and urban layouts automatically. Earlier research mainly focused on generating realistic city structures, while recent studies have also explored accessibility evaluation, land-use planning, and optimization.

Parish and Müller [1] introduced an L-system-based approach for generating roads, land subdivisions, and buildings using procedural rules. Chen et al. [2] proposed interactive street modeling using tensor fields, allowing users to generate and modify road networks. Benes et al. [3] studied procedural road-network generation by considering environmental and traffic-related factors. Nishida et al. [4] presented an example-driven method for producing road networks based on existing urban patterns.

Other studies focused on optimizing urban layouts for particular objectives. Pinnighoff et al. [5] investigated evolutionary computation for transport-network planning. Mustafa et al. [6] explored flood-sensitive urban layout generation using hydraulic simulation and optimization. Ortner et al. [7] examined generative urban modeling for walkable neighbourhoods, while Zheng et al. [8] applied deep reinforcement learning to spatial planning.

More recent research has also investigated road-network evaluation and urban interaction models. Dobbie and Johansson [9] studied network morphology and evaluation metrics for procedurally generated layouts. Santos et al. [10] proposed an agent-based approach that incorporates land-use and transport interactions.

These studies show that procedural generation can be applied to different urban planning objectives. However, the techniques vary in their complexity, input requirements, and evaluation methods. The proposed system adopts a simplified, deterministic city-generation approach and integrates road-graph construction, emergency accessibility evaluation, and greedy road optimization into an interactive application.

Table I summarizes the reviewed studies, their main objectives, techniques, and research gaps relevant to the proposed work.

Ref.	Paper / Year	Main focus	Method / Technique	Research gap
[9]	Comparing Urban Network Morphologies (2026)	Evaluation of generated road networks	CitySprout, L-systems, network metrics	Focuses on road morphology rather than complete urban complexity
[7]	Generative Urban Modelling for Walkable Neighbourhoods (2025)	Walkability-oriented urban design	Generative modeling, accessibility metrics, RBFOpt	Limited POI information and network discontinuities
[10]	Agent-Based Procedural City Generation (2022)	Land-use and transport interactions	Agent-based modeling, LUTI, Deep Q-Network	Computational complexity and simplified urban preferences
[6]	Flood-Sensitive Urban Layouts (2020)	Flood-risk reduction	Procedural generation, hydraulic simulation, MCMC	High computational cost and limited interactive control
[3]	Procedural Modelling of Urban Road Networks (2014)	Road-network generation	Procedural growth, environmental and traffic factors	Simplified traffic and land-use representation
[2]	Interactive Procedural Street Modeling (2008)	Interactive street generation	Tensor fields and graph editing	Emphasizes modeling rather than quantitative optimization
[4]	Example-Driven Procedural Urban Roads (2016)	Realistic road generation	Example-based and procedural graph growth	Depends on example networks and user guidance
[8]	Spatial Planning via Deep Reinforcement Learning (2023)	Automated urban planning	Graph Neural Network, Deep RL	Computationally demanding
[5]	Designing Road Networks Using Evolutionary Computation (2008)	Transport-network optimization	Genetic algorithms, evolutionary computation	Transport-focused rather than interactive visual analysis
[1]	Procedural Modeling of Cities (2001)	Foundational city generation	L-systems, global goals, local constraints	Rule complexity and limited functional evaluation

III. RESEARCH GAP AND CONTRIBUTION

Existing approaches often focus on sophisticated generation, specialized planning objectives, or computationally intensive optimization. The contribution of this study is not a new graph algorithm or a claim of globally optimal urban design. It is an accessible integration of procedural generation, graph-based emergency evaluation, incremental road improvement, and interactive visual inspection within one reproducible 2D workflow: Generate → Evaluate → Optimize → Compare

IV. IMPLEMENTED METHODOLOGY

A. User Parameters and Procedural Generation

The interface accepts a base seed, road density, building density, and one to five candidate layouts. The generator uses std::mt19937 with shuffled, bounded-jitter road slots to produce deterministic, full-boundary horizontal and vertical roads. Sorted road separators form rectangular cells, inset into blocks; approximately 40-unit lots are populated with randomized building footprints using a separate random stream. The default city boundary is (100,100)–(900,700), with 40-unit minimum road spacing. Road density ranges from 0 to 1 and building density from 0.1 to 1.

B. Road Graph and Facility Placement

Road endpoints and perpendicular crossings form coordinate-deduplicated graph vertices. Consecutive vertices on each road define unique, undirected edges weighted by Euclidean segment length.

$$w(u,v) = \sqrt{(x_v - x_u)^2 + (y_v - y_u)^2}$$

Where u and v are connected road-network nodes.

By default, three hospitals and four incidents are placed preferentially at building centers, followed by block or graph sites, and attached to the nearest road vertex. Access distances from facility positions to graph vertices are not included in route costs.

C. Reachability and Shortest-Path Evaluation

For each incident, queue-based breadth-first search (BFS) identifies hospitals reachable through the road graph, with

O(V+E) traversal complexity. A min-priority-queue implementation of Dijkstra's algorithm then computes weighted shortest paths to the reachable hospitals in O((V+E) log V) time, excluding hospital selection overhead. Predecessor links reconstruct the selected route. Equal-distance hospital ties favor the lower hospital identifier. An unreachable incident has no route and an infinite shortest-path distance.

D. Emergency Accessibility Score

The accessibility of a generated city is evaluated using two components: emergency reachability and shortest-road distance efficiency. For (N) emergency incidents and (K) reachable incidents, the reachability score (R) is calculated as

$$R = \frac{K}{N} \times 100$$

K = Number of emergency incidents that can reach a hospital.

N = Total number of emergency incidents.

R = Reachability percentage.

The distance-efficiency score (D) is calculated using the mean shortest-road distance (\bar{d}) of reachable incidents and the city-boundary diagonal (L), as expressed in (3).

$$D = 100\left(1 - \min\left(\frac{\bar{d}}{L}, 1\right)\right)$$

For the generated city boundary, (L=1000) world units. When no incident is reachable, (D=0). The overall accessibility score (S) combines both components using the weighted equation

$$S = 0.60R + 0.40D$$

The score is bounded between 0 and 100, with a value of zero when no incidents are present. The 60:40 weighting prioritizes reachability over distance efficiency and represents a project-specific heuristic rather than a calibrated emergency-service standard. Unreachable routes are excluded from the mean distance calculation, and all distances are measured in abstract world units rather than actual travel times.

E. Greedy Road Optimization

The deterministic greedy optimizer proposes full-span midpoint roads within available gaps, subject to perpendicular connection, spacing, and facility-clearance constraints. Each trial rebuilds the road graph, reattaches fixed-position facilities to their nearest nodes, reruns BFS and Dijkstra, and recalculates S. A proposal is accepted only if the score increases by more than 10^{-6}. Candidate proposals are regenerated after each acceptance; search stops on proposal exhaustion, attempt budget, or near-perfect score. The default attempt budget is 100, optimizer seed 42, spacing 40, and facility clearance 10. Only road addition is implemented; road deletion, relocation, and global-optimum search are not.

$$S_{new} > S_{current} + \varepsilon$$

F. Candidate Selection and Comparison

One to five candidates are generated from consecutive seeds and ranked by their original scores. The highest original score is identified, with lower candidate index breaking near-equal ties. Candidate A remains initially selected, and each candidate retains independent before/after optimization snapshots. Finite candidate comparison and greedy improvement are separate operations, neither implying an exhaustive search.

V. COMPUTER GRAPHICS IMPLEMENTATION

The system uses C++17, OpenGL 2.1 compatibility/fixed-function rendering, GLFW 3.4, bundled GLAD 0.1.36, Dear ImGui 1.91.9b and Windows Imaging Component for PNG decoding. glOrtho establishes a top-down Cartesian projection; glViewport and glScissor confine the city view with framebuffer/DPI-aware scaling. Immediate-mode quads, triangles, triangle fans, lines and points represent streets, property surfaces, buildings, canopies and graph overlays.

Building sprites use OpenGL texture upload, UV coordinates, RGBA alpha blending and aspect-preserving scaling. Translation is achieved by adding local offsets to world-space centers rather than explicit object transform matrices. Quarter-turn sprite orientation is achieved through dimension swapping and UV ordering rather than arbitrary-angle geometry rotation. Layers are drawn as ground/property paint, buildings, streets, optional graph, routes and facilities, followed by ImGui.

Nineteen PNGs are loaded (six residential, four commercial, four office, one hospital, and four tree images); however, tree canopies currently render as polygons, not tree sprites. Streets include asphalt, curbs, sidewalks, and dashed center markings; crosswalk bars are absent. OpenGL performs rasterization: custom DDA/Bresenham, 3D perspective, and shader-based rendering are not implemented.

VI. SYSTEM ARCHITECTURE

Simulation is separate from presentation. CityGenerator produces CityData; RoadGraphBuilder constructs RoadGraph; Emergency places hospitals/incidents; AccessibilityAnalyzer evaluates BFS reachability and Dijkstra routes; AccessibilityScorer computes the score; and CityOptimizer tests road additions. CandidateApplication manages generation, selection, and cached snapshots. VisualComposition, VisualAssets and StreetPresentation prepare decorative properties, building sprites and street profiles, while Renderer and ApplicationInterface display the city and controls.

Texture creation, drawing and destruction require an active OpenGL context, whereas graph analysis and optimization do not. Presentation envelopes and decorative surfaces do not

mutate simulation geometry. Optimization, by contrast, rebuilds road subdivisions and can change the building arrangement while preserving the physical facility coordinates within each before/after pair.

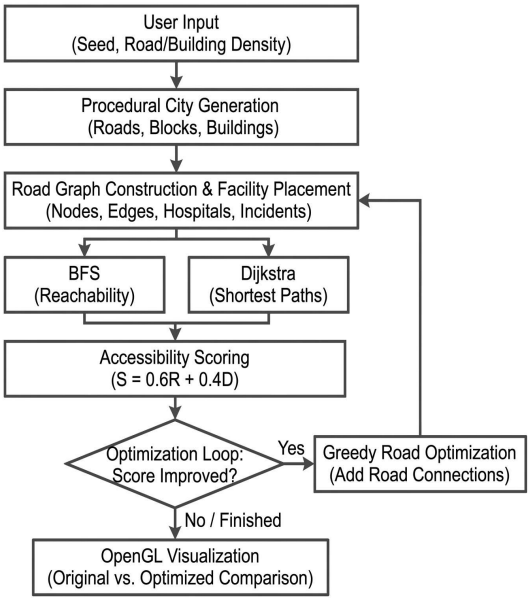


Fig 1. System Architecture of procedural city generator with layout optimization

VII. EXPERIMENTAL SETUP

The verification evaluated five predetermined city seeds (42–46), each with road density 0.50, building density 0.50, spacing 40, an 800 × 600 world-unit boundary, and default three hospitals/four incidents.

The greedy optimizer used seed 42 for every city, a 100-attempt budget and an acceptance threshold of 10^{-6}. Each seed had one warm-up and 21 fresh timed runs, with geometry, routes, scores, and optimization histories checked for repeatability. Comparisons were paired within seed because facilities can differ between independently generated cities.

The experiments used a Windows machine with Intel Core i5-12450HX and GCC 14.1.0. Timings were measured with std::chrono::steady_clock on Debug algorithm objects without compiler optimization.

Generation timing includes city geometry, graph, facilities and initial analysis; optimization timing includes attempted proposals and final rebuilding. Rendering, GUI, texture loading and process startup were excluded. The existing Debug build passed 12 of 12 CTest tests (0 failures; 16.15 seconds); graphical smoke tests used hidden OpenGL windows and synthetic input.



Fig. 2. Experimental environment showing the procedurally generated city layout, road network, and emergency facilities in the OpenGL application.

VIII. Comparison with Existing Procedural City Generation Methods

Previous studies have explored different techniques for procedural city generation, including L-systems, tensor fields, and example-driven road generation. Parish and Müller [1] used L-systems to generate road networks, land subdivisions, and buildings based on geographic and population information. Chen et al. [2] introduced tensor-field-based street modeling, allowing users to generate and interactively modify road networks. Nishida et al. [4] proposed an example-driven approach that generates road layouts using patterns extracted from existing road networks.

In comparison, the proposed system uses a deterministic, seed-based approach to generate horizontal and vertical roads, rectangular blocks, and buildings. Unlike methods designed to produce complex or realistic urban structures, the proposed generator focuses on reproducibility, adjustable road and building densities, and direct integration with graph-based accessibility analysis.

TABLE II. COMPARISON OF PROCEDURAL CITY GENERATION APPROACHES

Approach	Generation technique	Main characteristic
Parish and Müller [1]	L-system-based generation	Roads and buildings influenced by geographic data
Chen et al. [2]	Tensor-field-based modeling	Interactive street-network generation and editing

Nishida et al. [4]	Example-driven road growth	Generates road patterns inspired by existing examples
Proposed system	Seed-based grid generation	Reproducible layouts with integrated accessibility evaluation

The comparison highlights differences in generation objectives rather than establishing the superiority of one method. Existing approaches provide greater flexibility in representing complex street patterns, whereas the proposed system uses simplified geometric layouts to support repeatable experiments. Its additional functionality includes BFS-based reachability analysis, Dijkstra-based shortest-road evaluation, accessibility scoring, and greedy road optimization.

The current study does not reproduce the earlier generation algorithms or evaluate all approaches using a common dataset. Therefore, the comparison is qualitative, and no claim of higher generation accuracy or computational performance is made. A controlled benchmark using common road-network metrics would be required for a quantitative comparison.

IX. RESULTS AND DISCUSSION

Table I reports the five paired city experiments. All incidents were reachable before and after optimization (4/4 for every seed). Three seeds gained score through reduced shortest road distance; seeds 43 and 46 accepted no road and showed no improvement. Thus, the evidence supports improvements in the aggregate geometric accessibility objective for some generated cities, not improved reachability or reduced real-world ambulance response time.

TABLE III. PAIRED OPTIMIZATION RESULTS (WORLD UNITS)

Mean Distance	Score (before→after)	Roads/attempts
306.71 → 135.53	87.73 → 94.58	4/17
202.59 → 202.59	91.90 → 91.90	0/5
363.47 → 333.66	85.46 → 86.65	3/13
395.70 → 288.45	84.17 → 88.46	4/11
199.73 → 199.73	92.01 → 92.01	0/6

For seed 42, optimization added four roads over 17 attempts, increasing the score by 6.85 points and reducing mean shortest road distance by approximately 55.8%. Its road graph grew from 9 roads, 38 nodes, and 49 edges to 13 roads, 66 nodes, and 93 edges. Buildings changed from 91 to 84 because accepted roads trigger subdivision rebuilding. The original scores of candidate A (seed 42), B (seed 43) and C (seed 44) were 87.73, 91.90 and 85.46, respectively; B was the highest original-scoring candidate, even though A improved after separate optimization.

TABLE II. MEDIAN DEBUG CPU TIMINGS (21 RUNS; MILLISECONDS)

Generation + evaluation	Optimization
0.477	7.664
0.371	1.246
0.367	3.699
0.378	3.886
0.360	1.387

Table IV summarizes median computation times across 21 timed runs per seed. These measurements describe this specific Debug configuration and are not frame rates, Release-mode performance results or general scalability estimates. The observed improvements are limited to the project's normalized graph-distance metric. A higher aggregate score does not guarantee that every individual incident route improves.

X. LIMITATIONS

The proposed system has certain limitations due to its simplified city model and optimization approach. The procedural generation process primarily uses axis-aligned roads and rectangular blocks, which may not represent the complexity of real-world urban layouts. Emergency accessibility is evaluated using shortest road distances rather than actual travel times, as factors such as traffic congestion, vehicle speed, road restrictions, and hospital capacity are not considered. Furthermore, the greedy optimization algorithm only adds road connections when the overall accessibility score improves and does not guarantee a globally optimal solution. The scoring model uses fixed weights of 60% for reachability and 40% for distance efficiency, which have not been calibrated using real-world emergency service data. Additionally, experimental evaluation was limited to five city seeds under a single density configuration, restricting conclusions about scalability and performance across different urban environments. Therefore, the system is primarily intended as an educational and experimental tool rather than a practical urban planning solution.

XI. FUTURE SCOPE

Potential extensions include real geographic data import, travel-time-aware road weights, population-based emergency demand, constrained road construction cost, comparison of multiple optimization methods, and three-dimensional visualization. Such extensions would require additional models and validation; they are not implemented in the current system.

XII. CONCLUSION

This work presents an implemented 2D procedural city generator integrating deterministic city creation, weighted road-graph analysis, BFS reachability, Dijkstra shortest paths, a normalized accessibility score, and add-only greedy optimization. In a controlled five-seed experiment, three cities improved their aggregate accessibility scores, while two remained unchanged; all retained full reachability. The largest observed gain, for seed 42, was 6.85 score points, with mean road distance decreasing from 306.71 to 135.53 world units. The system provides a reproducible educational demonstration of how road layout affects graph-based accessibility, without claiming real-world emergency response-time improvements.

Code Availability

Code is available at Github:

https://github.com/khushicomp/Fa2_ProceduralGenerator

REFERENCES

[1] Y. I. H. Parish and P. Müller, “Procedural modeling of cities,” in Proc. 28th Annu. Conf. Comput. Graph. Interactive Techniques (SIGGRAPH), 2001, pp. 301–308, doi: 10.1145/383259.383292.

[2] G. Chen, G. Esch, P. Wonka, P. Müller, and E. Zhang, “Interactive procedural street modeling,” ACM Trans. Graph., vol. 27, 2008.

[3] J. Beneš, A. Wilkie, and J. Křivánek, “Procedural modeling of urban road networks,” Computer Graphics Forum, vol. 33, pp. 132–142, 2014, doi: 10.1111/cgf.12283.

[4] G. Nishida, I. Garcia-Dorado, and D. G. Aliaga, “Example-driven procedural urban roads,” Computer Graphics Forum, vol. 35, no. 6, pp. 5–17, 2016, doi: 10.1111/cgf.12728.

[5] M. Pinninghoff, R. Contreras, and J. Atkinson, “Designing road networks for transport planning using evolutionary computation,” Applied Artificial Intelligence, vol. 22, no. 9, pp. 921–936, 2008, doi: 10.1080/08839510802296077.

[6] A. Mustafa et al., “Procedural generation of flood-sensitive urban layouts,” Environment and Planning B: Urban Analytics and City Science, vol. 47, no. 5, pp. 889–911, 2020, doi: 10.1177/2399808318812458.

[7] F. P. Ortner, Z. Chen, E. E. Aydin, and P. Song, “Generative urban modelling for walkable neighbourhoods: Design optimization for pedestrian-oriented street networks,” Journal of Urban Design, pp. 515–534, 2025, doi: 10.1080/13574809.2025.2517652.

[8] Y. Zheng, Y. Lin, L. Zhao, T. Wu, D. Jin, and Y. Li, “Spatial planning of urban communities via deep reinforcement learning,” Nature Computational Science, vol. 3, pp. 748–762, 2023, doi: 10.1038/s43588-023-00503-5.

[9] C. Dobbie and A. Johansson, “Comparing urban network morphologies: A new living metric and a framework for evaluating procedurally generated city layouts,” EPJ Data Science, vol. 15, Art. no. 76, 2026, doi: 10.1140/epjds/s13688-026-00694-7.

[10] L. F. S. E. dos Santos, C. Aranha, and A. P. de L. F. de Carvalho, “An agent-based approach to procedural city generation incorporating Land Use and Transport Interaction models,” in Proc. XIX Encontro Nacional de Inteligência Artificial e Computacional (ENIAC), 2022, pp. 246–257, doi: 10.5753/eniac.2022.227605.